

Algoritmi e Strutture Dati – 17/01/2022 – Parte A

Esercizio A1 – Complessità – Punti ≥ 8

Trovare i limiti superiore e inferiore più stretti possibili per la seguente equazione di ricorrenza:

$$T(n) = \begin{cases} 1 & n < 8 \\ 2T(\lfloor n/2 \rfloor) + 31T(\lfloor n/8 \rfloor) + n^2 - n & n \geq 8 \end{cases}$$

Esercizio A2 – Alberi made up – Punti ≥ 10

Un albero binario *ma-come-se-li-inventa* è un albero in cui:

- i nodi che si trovano in livelli dispari hanno esattamente 1 figlio
- i nodi che si trovano in livelli pari hanno 0 o 2 figli.

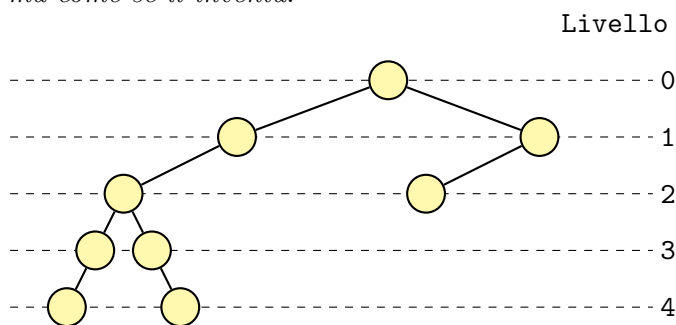
Vi ricordo che la radice si trova nel livello 0, i suoi figli si trovano nel livello 1, i suoi nipoti nel livello 2, ecc.

Scrivere un algoritmo

boolean isMakeup(TREE *T*)

che prende in input un albero binario *T* non vuoto e restituisce **true** se *T* è un albero *ma-come-se-li-inventa*, **false** altrimenti.

Per esempio, l'albero seguente è un albero binario *ma-come-se-li-inventa*.



Discutere informalmente la correttezza dell'algoritmo e la sua complessità computazionale.

Esercizio A3 – Serie aritmetica bucata – Punti ≥ 12

Si consideri una progressione aritmetica

$$x, x + d, x + 2d, \dots, x + nd$$

composta da $n + 1$ termini, con $n \geq 2$ e $d > 0$.

Supponiamo che venga memorizzata, in ordine, in un vettore *A* di n elementi, omettendo di memorizzare uno dei termini all'interno della serie (in altre parole, x e $x + nd$ sono sicuramente presenti).

Scrivere un algoritmo

int missing(**int**[] *A*, **int** *n*)

che prende in input il vettore *A* descritto sopra e restituisce il termine mancante.

Spiegare il funzionamento e discutere la complessità computazionale dell'algoritmo proposto. *Soluzioni in tempo lineare o superiore non verranno considerate.*

Per esempio, se $A = [3, 5, 7, 9, 11, 15, 17]$, l'algoritmo deve restituire 13.